当前 TODO：

- 修正生成 output 的逻辑**【进行中】**
- Design Rules（全部手绘时考虑 or 全部参数检查）**【进行中】**
- sram 的测试如何受 rc 影响**【还没做】**
- 同步扩展 1-2 个结构（反相器，NAND 等）验证全流程的泛化能力**【进行中】**

其他归档的 todo：

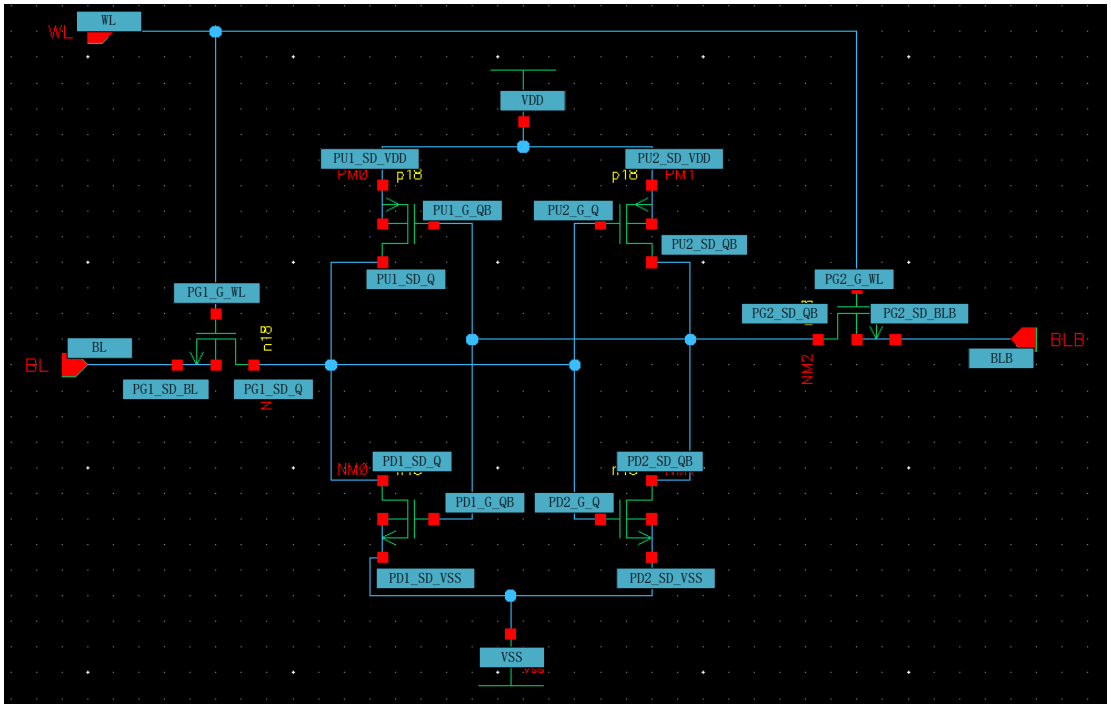
- IV 校准**【已完成】**
- RC 提取到电路**【已完成】**
- Hspice test 出结果,Output**【已完成】**

阶段性进展：

【如何设计通用的 pin 文件——兼顾泛化能力和可读性】

对于人类而言，对电路图的理解主要来自于类似 virtuoso 的 schematic 和 layout，人类对抽象的无序的电路很容易没有概念，因此为了让人能够更多地参与和验证网表的正确性和可行性，就需要更多地从 schematic 和 layout 获取信息。

结合前几次的情况分析，我们已经有了可以泛化的 schematic 方案，如下图所示：



为了让这个命名模式更加稳定，同时增加人的可交互性，我们采用了 pin 模板文件作为某个标准电路的模板：

在这个模板中（R）部定义了电路中全部的 contact 点，（C）部定义了电路中全部的金属引线，（FET）部定义了全部 FET 及各自的 S,G,D,B 端对应的 contact 点，并根据 L=default W=default NF=default 确定可更改的 L，W 和 NF（个人认为并不建议修改，修改该这里的 L 和 W 没有意义，不过对于一些电路测试而言，临时的 L 和 W

的改动应该是一个比较常见的设置)。(PORT)是标准单元的外部引出接口。

```
./INPUT_MODEL/{env:NAME}_pin.txt
(R){WL,PG1_G_WL,PG2_G_WL,BL,PG1_SD_BL,BLB,PG2_SD_BLB,VDD,PU1_S
D_VDD,PU2_SD_VDD,VSS,PD1_SD_VSS,PD2_SD_VSS,PG1_SD_Q,PU1_SD_Q,P
D1_SD_Q,PU2_G_Q,PD2_G_Q,PG2_SD_QB,PU2_SD_QB,PD2_SD_QB,PU1_G_QB
,PD1_G_QB}
R_WL_PG1_G_WL
R_WL_PG2_G_WL
R_BL_PG1_SD_BL
R_BLB_PG2_SD_BLB
R_VDD_PU1_SD_VDD
R_VDD_PU2_SD_VDD
R_VSS_PD1_SD_VSS
R_VSS_PD2_SD_VSS
R_PG1_SD_Q_PU1_SD_Q
R_PG1_SD_Q_PD1_SD_Q
R_PG1_SD_Q_PU2_G_Q
R_PG1_SD_Q_PD2_G_Q
R_PG2_SD_QB_PU2_SD_QB
R_PG2_SD_QB_PD2_SD_QB
R_PG2_SD_QB_PU1_G_QB
R_PG2_SD_QB_PD1_G_QB
(C){VDD,VSS,BL,BLB,WL,Q,QB}
C_VDD_VSS
C_VDD_BL
C_VDD_BLB
C_VDD_WL
C_VDD_Q
C_VDD_QB
C_VSS_BL
C_VSS_BLB
C_VSS_WL
C_VSS_Q
C_VSS_QB
C_BL_BLB
C_BL_WL
C_BL_Q
C_BL_QB
C_BLB_WL
C_BLB_Q
```

```
C_BLB_QB
C_WL_Q
C_WL_QB
C_Q_QB

(FET){N1,N2,N3,N4,P1,P2}{S,G,D,B}
N1 S=PG1_SD_Q   G=PG1_G_WL  D=PG1_SD_BL   B=VSS L=default
W=default NF=default
N2 S=PG2_SD_QB  G=PG2_G_WL  D=PG2_SD_BLB  B=VSS L=default
W=default NF=default
N3 S=PD1_SD_VSS G=PD1_G_QB  D=PD1_SD_Q    B=VSS L=default
W=default NF=default
N4 S=PD2_SD_VSS G=PD2_G_Q   D=PD2_SD_QB   B=VSS L=default
W=default NF=default
P1 S=PU1_SD_VDD G=PU1_G_QB  D=PU1_SD_Q    B=VDD L=default
W=default NF=default
P2 S=PU2_SD_VDD G=PU2_G_Q   D=PU2_SD_QB   B=VDD L=default
W=default NF=default

(PORT){BL,BLB,WL,VDD,VSS,Q,QB}
BL=BL
BLB=BLB
WL=WL
VDD=VDD
VSS=VSS
Q=PG1_SD_Q
QB=PG2_SD_QB
```

这样做（创建一个输入的_pin.txt）的意义是什么呢？这个是拉齐版图-sde_tdr-hspice 的最后一块拼图。要保证在 spice 中的端口名称和版图设置、sde-tdr 保持一致就必须从定义上保持命名一致，并且让脚本能够精确识别到是哪些端口需要 R 和 C 进入后道互联的过程中（如下图所示，特别是对于未作为 contact 的 Q 和 QB 通过检测点的方式获取 `PD1_SD_Q -> Q`），刚才 default 的 `L=1.6e-8 W=3.5e-8 NF=1` 也正确显示了出来。功能上这个

```
./Hspice/{env:NAME}_pin_spice.txt
* AutoLDM data-driven extracted-RC subcircuit library
.subckt TEST_PEX BL BLB WL VDD VSS Q QB
```

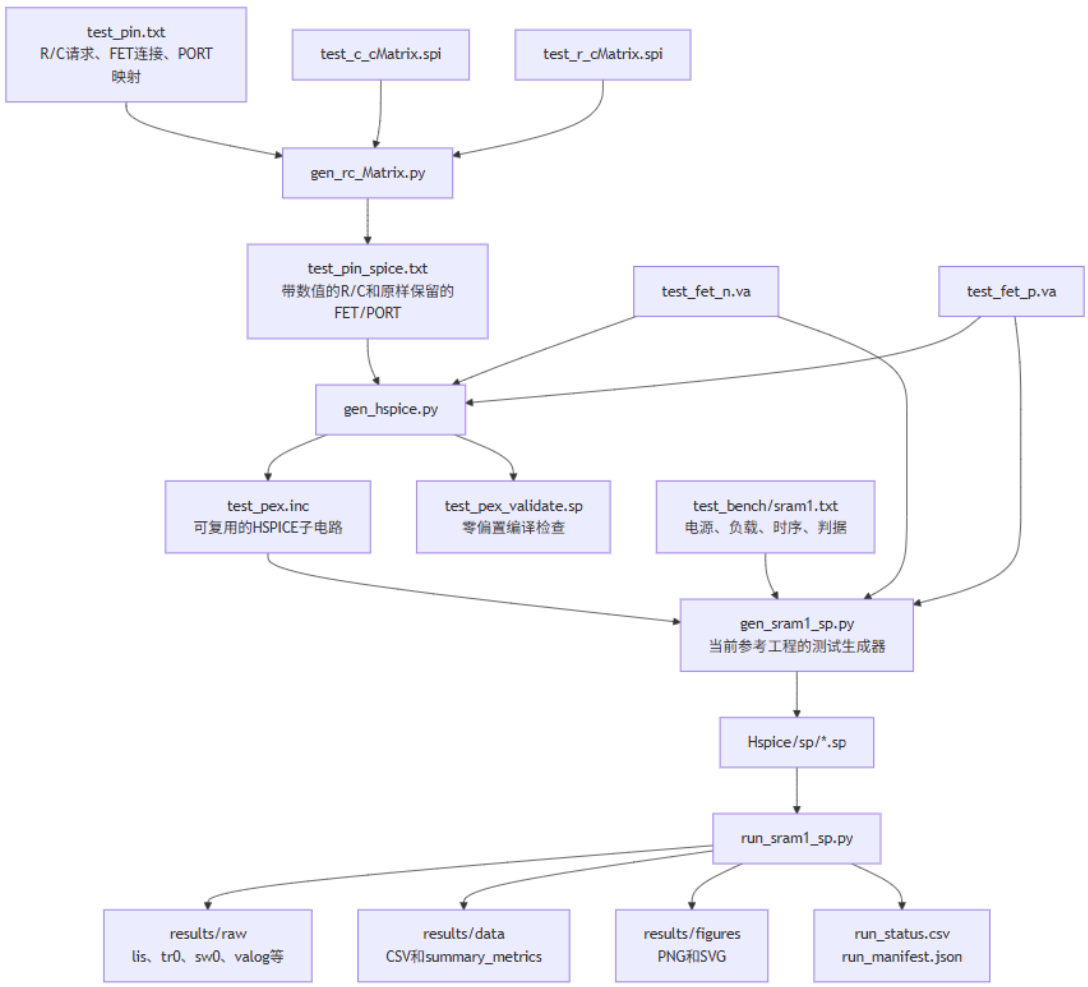
```
* Canonical aliases from PORT mappings and zero-ohm R:
*   PD1_SD_Q -> Q
*   PD2_SD_QB -> QB
*   PG1_SD_Q -> Q
*   PG2_SD_QB -> QB
R_WL_PG1_G_WL WL PG1_G_WL 1.94588312e+1
R_WL_PG2_G_WL WL PG2_G_WL 1.94591882e+1
R_BL_PG1_SD_BL BL PG1_SD_BL 3.72184969e+0
R_BLB_PG2_SD_BLB BLB PG2_SD_BLB 2.04688660e+1
R_VDD_PU1_SD_VDD VDD PU1_SD_VDD 2.06153400e+1
R_VDD_PU2_SD_VDD VDD PU2_SD_VDD 2.06153400e+1
R_VSS_PD1_SD_VSS VSS PD1_SD_VSS 1.90815000e+1
R_VSS_PD2_SD_VSS VSS PD2_SD_VSS 1.90815000e+1
R_PG1_SD_Q_PU1_SD_Q Q PU1_SD_Q 5.39327410e+1
R_PG1_SD_Q_PU2_G_Q Q PU2_G_Q 6.64110757e+1
R_PG1_SD_Q_PD2_G_Q Q PD2_G_Q 3.97463296e+1
R_PG2_SD_QB_PU2_SD_QB QB PU2_SD_QB 5.59725865e+1
R_PG2_SD_QB_PU1_G_QB QB PU1_G_QB 6.67286046e+1
R_PG2_SD_QB_PD1_G_QB QB PD1_G_QB 4.00123334e+1
C_VDD_VSS VDD VSS 1.76277100e-20
C_VDD_BL VDD BL 2.55341808e-20
C_VDD_BLB VDD BLB 2.49318642e-20
C_VDD_WL VDD WL 4.66267270e-18
C_VDD_Q VDD Q 1.06102143e-17
C_VDD_QB VDD QB 1.05952666e-17
C_VSS_BL VSS BL 2.54566470e-18
C_VSS_BLB VSS BLB 2.28498501e-18
C_VSS_WL VSS WL 3.15065429e-17
C_VSS_Q VSS Q 1.41671393e-17
C_VSS_QB VSS QB 1.45239211e-17
C_BL_BLB BL BLB 6.86933590e-20
C_BL_WL BL WL 1.79141723e-17
C_BL_Q BL Q 3.25662978e-18
C_BL_QB BL QB 4.28822112e-19
C_BLB_WL BLB WL 1.79572401e-17
C_BLB_Q BLB Q 9.78514812e-19
C_BLB_QB BLB QB 3.00407779e-18
C_WL_Q WL Q 3.33207273e-17
C_WL_QB WL QB 5.19794056e-17
C_Q_QB Q QB 9.33229610e-17
XN1 PG1_SD_BL PG1_G_WL Q VSS nmos_lvt L=1.6e-8 W=3.5e-8 NF=1
XN2 PG2_SD_BLB PG2_G_WL QB VSS nmos_lvt L=1.6e-8 W=3.5e-8 NF=1
```

```
XN3 Q PD1_G_QB PD1_SD_VSS VSS nmos_lvt L=1.6e-8 W=3.5e-8 NF=1
XN4 QB PD2_G_Q PD2_SD_VSS VSS nmos_lvt L=1.6e-8 W=3.5e-8 NF=1
XP1 PU1_SD_Q PU1_G_QB PU1_SD_VDD VDD pmos_lvt L=1.6e-8 W=3.5e-
8 NF=1
XP2 PU2_SD_QB PU2_G_Q PU2_SD_VDD VDD pmos_lvt L=1.6e-8 W=3.5e-
8 NF=1
.ends TEST_PEX
```

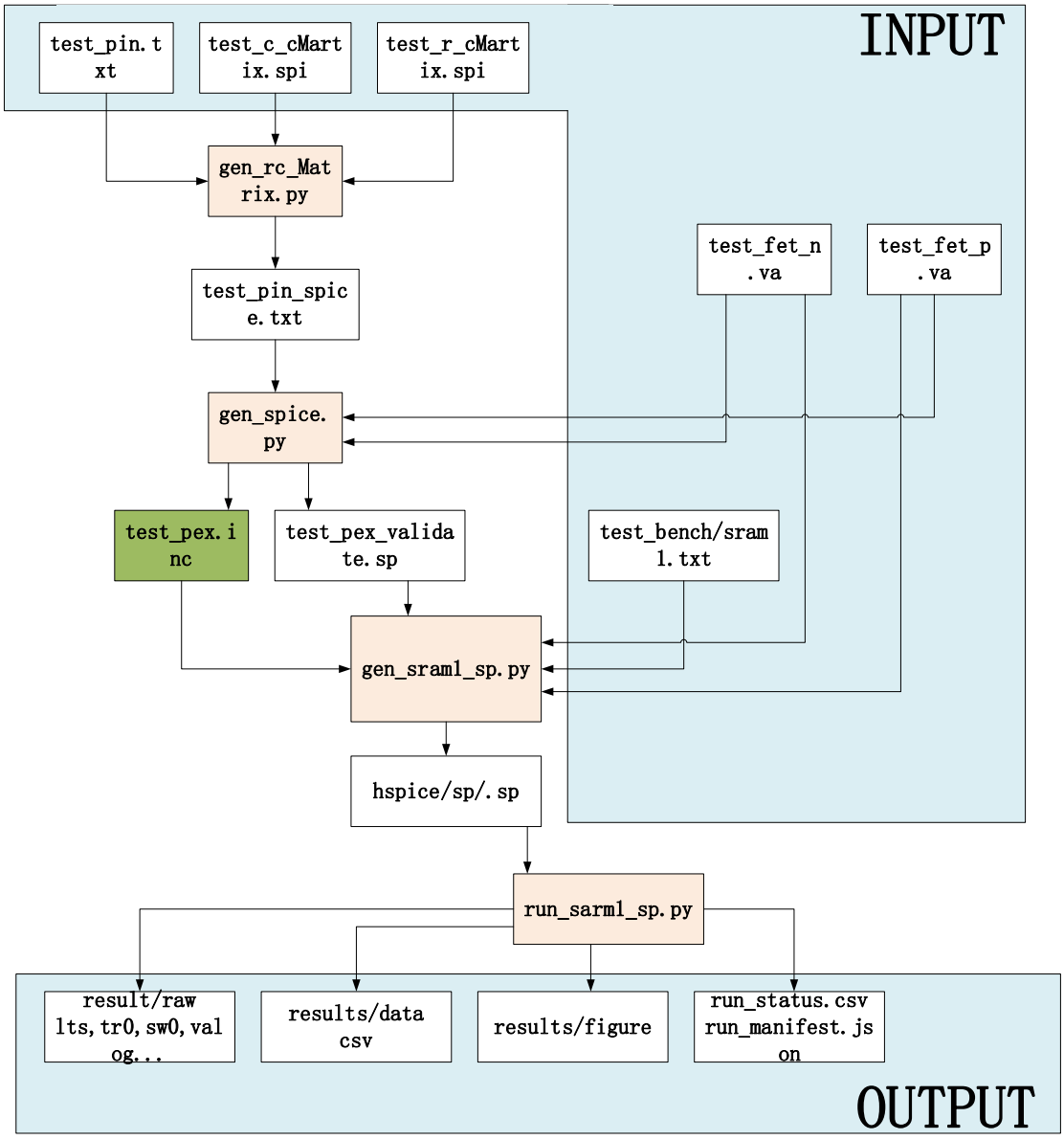
【hspice 部分的文件层级和脚本设计】

本部分全部内容已经有完整的文件（包括输出结果），但是因为不是最终版就先讲一下当前的优化思路。在最后的附件有少量图片（不作为可参考的有效数据）。

初版（如图所示）：



gen_rc_Matrix.py 通过 sde 生成的 spi 和 test_pin 文件处理得到经过处理的 test_pin_spice，此时结合 nfet 和 pfet 的 va 文件，通过 gen_spice 脚本能够得到可以复用的 inc 文件，同时针对刚生成的 inc 进行一次 smoke_test。根据生成的 inc 和 test_bench 设置文件（这个文件只定义 vdd 和 vss 的电压，其他要把什么放进来还没有想好），经过 gen_sram1_sp.py（全称是生成 sp 测试脚本-sram1 号脚本），顾名思义，这个脚本需要根据电路差异和关注的参数差异调整。因此针对不同标准单元我们应该出一整套完整的库文件，以适应不同的测试条件和测试单元，这部分可以通过 ai 的方式批量生成，但是仍然可以作为一个标准的方案进行固定（同时应当给出具体的 skill 能够直接生成）。然后后面的过程就可以通过 hspice 自动进行了。

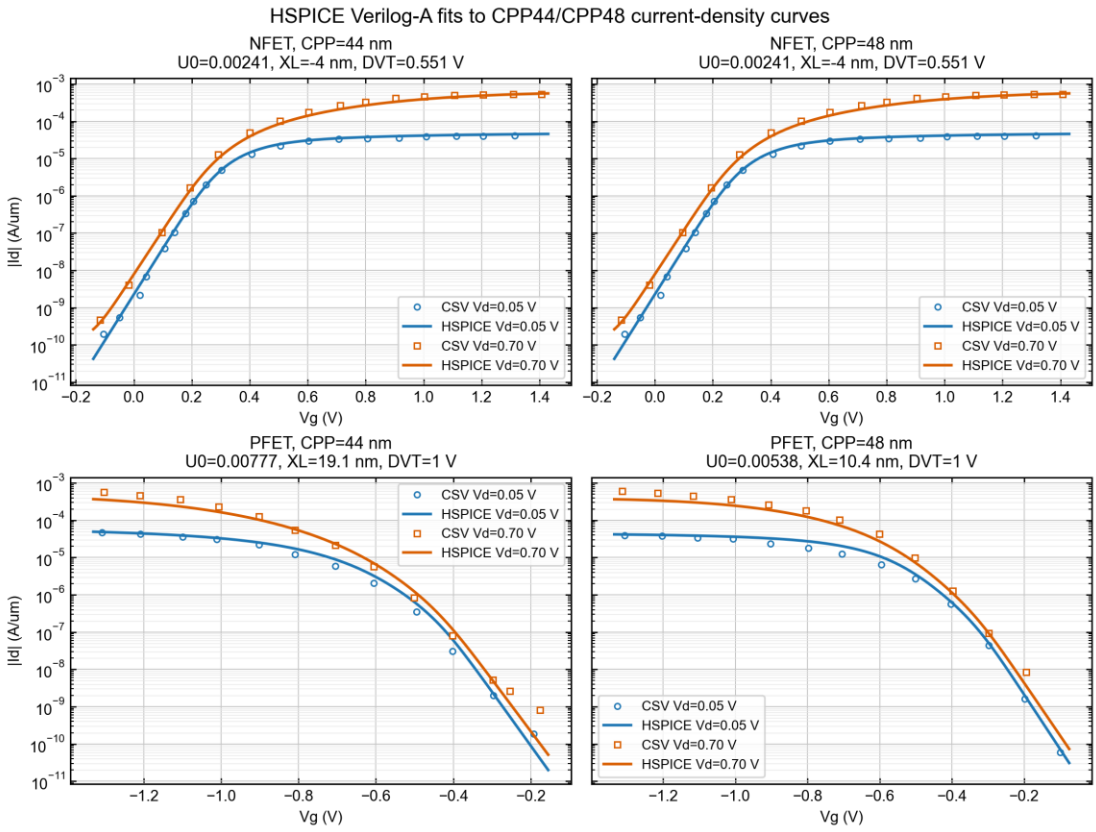


重新画了一下，可以看到，在流程里面的中间脚本很复杂（这里复杂指重复多），同时中间得到的文件又大多数没什么用。绿色部分标出的可以认为是针对这个流程一个比较重要的中间量。下一步的修改目标是将 gen_rc.py，gen_spice.py 进行融合，再将 gen_sram1_sp 和 run_sram1_sp 进行融合，同时通过一些比较明确和规范的定义（绝对路径），确定 hspice 可执行文件的位置。**这四个脚本最终可以合并成两个脚本【这个内容可能还要综合模型特点进行修改】。**

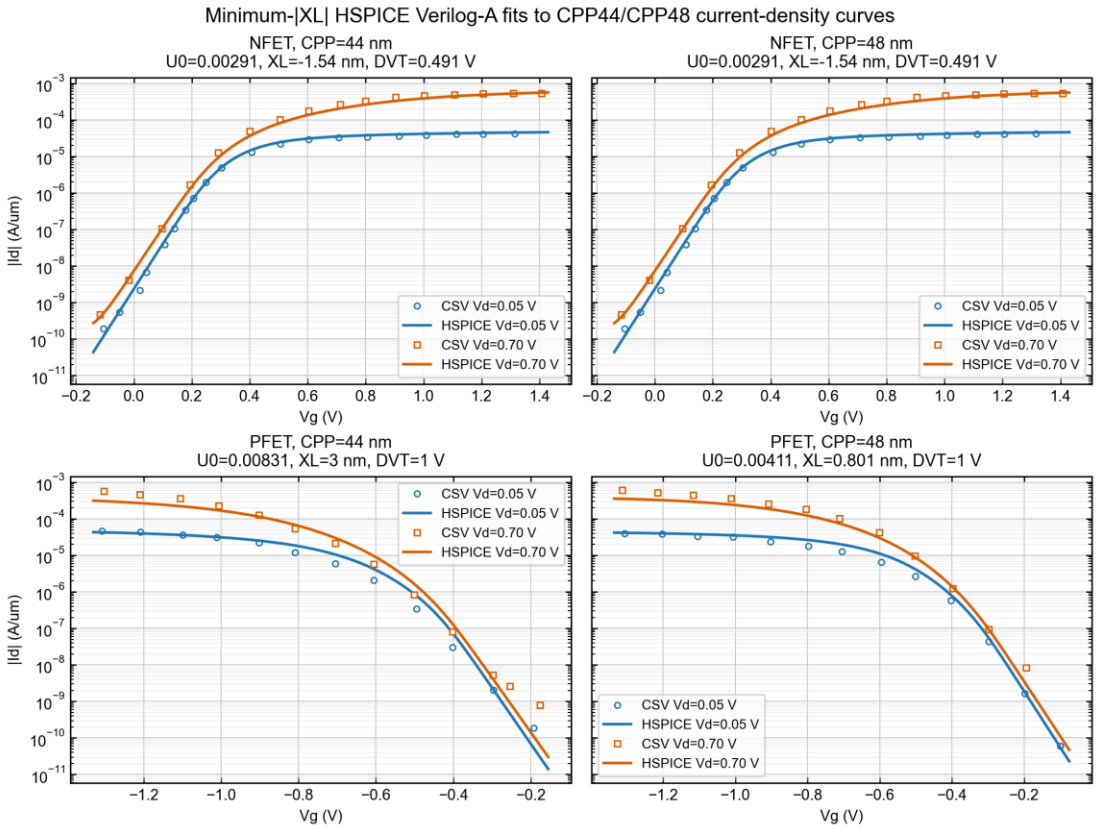
【拟合】

拟合需要同时处理 N/P 两类器件、CPP44/CPP48 两个工艺角，以及 Vd=0.05 V 和 0.70 V 两组曲线，一共 8 条 Id-Vg 曲线。每试一组参数都生成一组 HSPICE deck、运行仿真、读取电流并换算到 A/um，再计算和拟合标准之间的误差。

VA 中保持 L、W、NF、TGAA、NGAA、NBODY、NSD 和 PHIG 不变，实际调整 U0、XL、DVTSHIFT、DeltaWGAA、DeltaTGAA 和 EOT_0。先做了一套以曲线误差为主的拟合，得到下图结果。N 型两组 CPP 数据重合，因此复用同一组拟合参数不会影响曲线；P 型高漏压曲线更难同时兼顾，目前仍有一点差距。

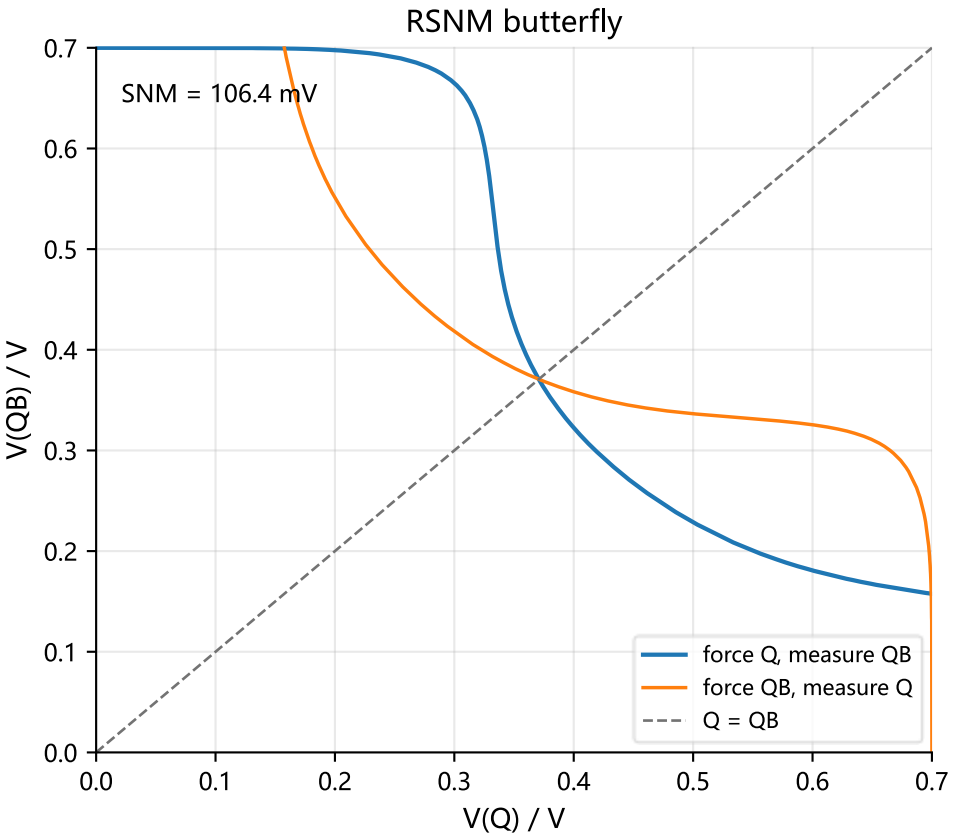


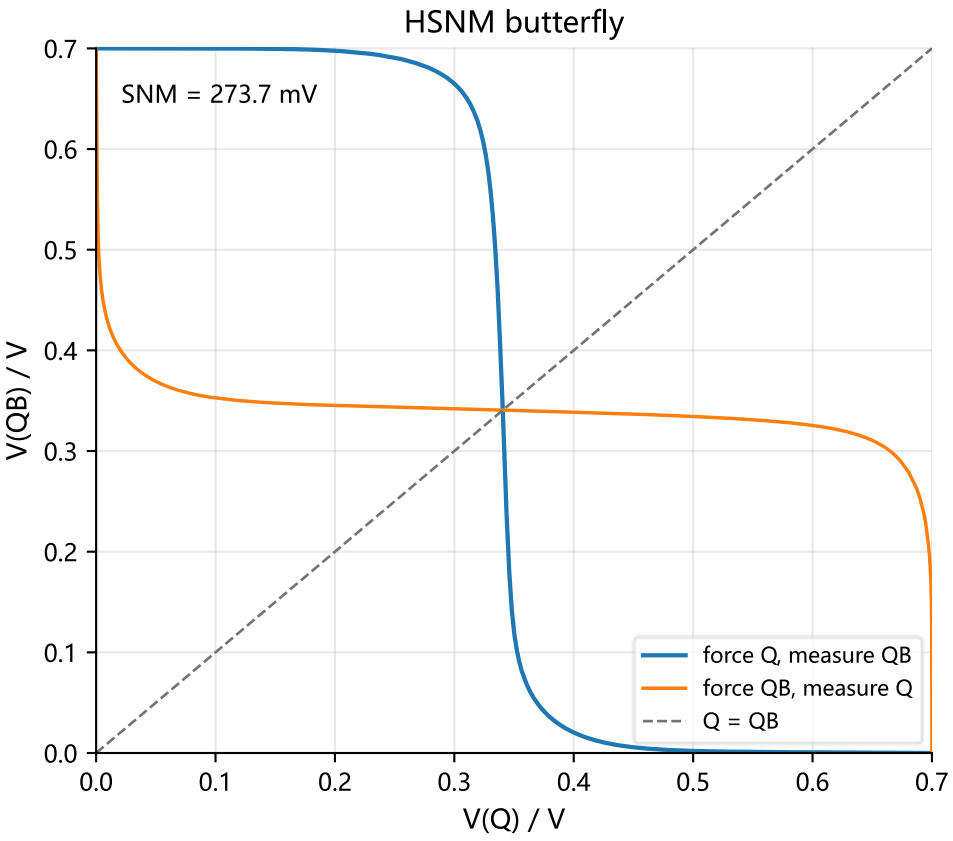
第一套结果把整体曲线拟合得更好，但 P 型器件的 XL 分别到了约 19.1 nm 和 10.4 nm，数值偏大。为了尽可能控制$|XL|/L$尽可能小，又在保留第一套结果的前提下增加了|XL|不超过 3 nm 的约束，重新跑了一套拟合，结果如下（整体看起来结果可以接受）。



目前已经保留两套完整结果：一套优先拟合误差，一套优先限制 XL；每套都包含 CPP44/CPP48 的完整 N/P VA 模型、JSON 报告和对比图。现在完成的是 DC Id-Vg 拟合，电荷、电容和瞬态还没有用数据校准。

【附件-结果展示】


RSNM butterfly
SNM = 106.4 mV
V(QB) / V
V(Q) / V
force Q, measure QB
force QB, measure Q
Q = QB
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7


HSNM butterfly
SNM = 273.7 mV
V(QB) / V
V(Q) / V
force Q, measure QB
force QB, measure Q
Q = QB
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7